

Ethics as a Service in Service-oriented Software Engineering

Part 1 - AI Basics and Machine Ethics Foundations

Embedded Special Topic for Service-oriented Software Engineering

University of L'Aquila

Patrizio Migliarini, PhD

AI basics, what AI is, what it is not, why ethics enters the picture

Course framing

Ethics as a Service in Service-oriented Software Engineering

From AI basics to operational governance

Why ethics must become usable, not only admirable

AI basics, what AI is, what it is not, why ethics enters the picture

What AI is

AI is a set of technologies that perform tasks usually associated with human intelligence

Typical tasks:

- learning from data
- recognising patterns
- understanding language
- supporting or automating decisions

AI basics, what AI is, what it is not, why ethics enters the picture

What AI is not

AI does not think in the human sense

AI does not have feelings

AI does not have moral consciousness

Its outputs come from statistical and computational processes

AI basics, what AI is, what it is not, why ethics enters the picture

What sits underneath AI

- Data
- Training
- Model
- Deployment context
- Human choices at every stage

<https://teachablemachine.withgoogle.com/>

AI basics, what AI is, what it is not, why ethics enters the picture

Why data matter ethically

Training data shape system behaviour

Biased data can produce biased outputs

Historical patterns can become automated injustice

Technical quality does not guarantee ethical quality

AI basics, what AI is, what it is not, why ethics enters the picture

Typical harms already observed

recruiting discrimination (amazon recruiting gender bias)
(<https://www.reuters.com/article/us-amazon-com-jobs-automation-insight-idUSKCN1MK08G>)

biased healthcare prioritisation (US healthcare black people discrimination)
(<https://www.science.org/doi/10.1126/science.aax2342>)

unfair credit allocation (Apple Card gender discrimination)
(<https://www.bbc.com/news/business-50365609>)

facial recognition misidentification (US police black people discrimination)
(<https://www.media.mit.edu/projects/gender-shades/overview/>)

<https://theyseeyourphotos.com/>

AI basics, what AI is, what it is not, why ethics enters the picture

Why ethics appears as soon as autonomy appears

autonomy means decision-making power over goals or actions

more autonomy means more ethically significant consequences

software can affect rights, opportunities, safety, and dignity

AI basics, what AI is, what it is not, why ethics enters the picture

If an AI system causes harm, who is responsible?

- the developer?
- the deployer?
- the data curator?
- the manager?
- the user?
- the organisation?



AI basics, what AI is, what it is not, why ethics enters the picture

Is that a matter of culture?



AI basics, what AI is, what it is not, why ethics enters the picture

Is that a matter of culture?

<https://www.moralmachine.net/>

AI basics, what AI is, what it is not, why ethics enters the picture

Is that a matter of culture?

East Asia: More likely to save the elderly

Latin America/French-speaking countries: Preference for young people, women, and healthy people

Countries with high inequality (e.g., Colombia): Prioritize individuals of high social status

Germany, Japan: Punish those who violate the rules (e.g., pedestrians walking outside the crosswalk)

<https://www.nature.com/articles/s41586-018-0637-6/>



Machine ethics foundations and the first practical workshop

What is machine ethics?

Machine ethics asks how machine behaviour should be constrained, guided, or evaluated

It is not the same as AI capability research

It is about morally significant behaviour in socio-technical systems

Machine ethics foundations and the first practical workshop

Three major ethical lenses

- Consequentialism
- Deontology
- Virtue ethics

Machine ethics foundations and the first practical workshop

Consequentialism in simple terms

- judge actions by consequences
- asks: what outcomes follow?
- asks: for whom?
- asks: over what time horizon?

Machine ethics foundations and the first practical workshop

Utilitarianism as the best-known consequentialism

- maximise overall well-being, happiness, or utility
- often attractive for optimisation
- difficult because utility is not simple or neutral

Machine ethics foundations and the first practical workshop

Deontology in simple terms

- judge actions by duties, rules, or rights
- some actions are wrong even if useful
- constraints matter

Machine ethics foundations and the first practical workshop

Virtue ethics in simple terms

- judge actions by character and practical wisdom
- ask what a good or trustworthy agent would do
- context matters

Machine ethics foundations and the first practical workshop

Why no single lens is enough

- consequences matter
- rules matter
- character and judgement matter
- real systems often involve conflict between them

Machine ethics foundations and the first practical workshop

Workshop 1: analyse a simple dilemma

Case: autonomous vehicle must choose between two harmful outcomes

Analyse the case through:

- Consequentialism
- Deontology
- Virtue ethics

Machine ethics foundations and the first practical workshop

Debriefing

Compare and defend the conclusions!

Debriefing

Ethical disagreement is not a failure of analysis.

It is part of the design problem!



Ethics as a Service in Service-oriented Software Engineering

Part 2 - From principles to operationalisation

Embedded Special Topic for Service-oriented Software Engineering

University of L'Aquila

Patrizio Migliarini, PhD

From principles to operationalisation

Why principles are not enough

- Fairness
- Privacy
- Accountability
- Transparency
- Safety

all sound right

none tells an engineer exactly what to do



Morley et al. - Ethics as a Service: A Pragmatic Operationalisation of AI Ethics

<https://link.springer.com/article/10.1007/s11023-021-09563-w>

From principles to operationalisation

The theory-practice gap

- from “what matters” to “how to build”
- abstract ethics versus design choices
- the central operational problem

From principles to operationalisation

The first naive solution: translational tools

- Checklists
- Scorecards
- Fairness libraries
- Audit templates
- Ethics canvases
- Impact assessments

From principles to operationalisation

Too flexible, too strict

too flexible:

- enables ethics washing
- enables ethics shopping

too strict:

- ignores context
- freezes contested values

From principles to operationalisation

Why one-off ethics checks fail

ethics cannot be a single checkbox, we need

- Validation
- Verification
- Evaluation

repeated reflection is required

From principles to operationalisation

A more promising direction: reflective procedure

- procedural regularity
- public reason
- contextual interpretation
- transparent trade-offs
- repeated review

From principles to operationalisation

Workshop 2: break an ethics checklist

A company uses an AI system to pre-screen internship applications for software engineering roles.

The company says the **system is ethical** because:

- gender and ethnicity are not explicit input fields
- the same threshold is applied to all candidates
- a fairness review has been completed
- a recruiter can still review flagged applications
- the system improves speed and consistency

Try to “break” the canvas by showing:

- where the company can appear compliant
- where important context is hidden
- where fairness is defined too narrowly
- where the process can be gamed
- where responsibility is blurred

Ethical canvas

System and decision - What decision is the system influencing or automating?

Context of use - In what setting is it used, with what stakes, constraints, and urgency?

Affected stakeholders - Who benefits, who bears risk, who may be excluded, who may remain invisible?

Data and labels - What data are used, who produced them, and what historical bias may be embedded?

Protected groups and proxies - Which groups matter, and which variables may act as hidden proxies?

Fairness lens chosen - What exactly counts as fairness in this case, and why this interpretation?

Error distribution - Who is harmed by false positives and false negatives?

Oversight and contestability - Who can question, override, appeal, or stop the system?

Evidence and monitoring - What evidence would show failure after deployment?

Open tensions - What ethical disagreement remains unresolved?

From principles to operationalisation

Debriefing

- **where the company can appear compliant**
 - Protected attributes removed*
 - Same threshold for all*
 - Fairness review completed*
 - Human recruiter still involved*
- **where important context is hidden**
 - Historical hiring bias becomes training data*
 - Unequal access exists before scoring*
 - The real decision is who becomes visible*
- **where fairness is defined too narrowly**
 - Fairness = same threshold only*
 - False negatives ignored*
 - Unequal opportunity ignored*
- **where the process can be gamed**
 - Proxies remain: postcode, university, career gaps*
 - Fairness checked only after filtering*
 - Human review may be superficial*
- **where responsibility is blurred**
 - Recruiter blames the tool*
 - Developers blame the data*
 - Management blames the vendor*
 - Nobody owns the unfair outcome*

From principles to operationalisation

Transition to EaaS

We still need principles

We still need tools

But we also need:

- **Process**
- **Context**
- **Oversight**
- **Accountability**

This is where EaaS begins



Ethics as a Service in Service-oriented Software Engineering

Part 3 – EaaS as a bounded service block

Embedded Special Topic for Service-oriented Software Engineering

University of L'Aquila

Patrizio Migliarini, PhD

Ethics as a Service as a bounded service block

What EaaS is trying to solve

Make AI ethics usable for practitioners

Avoid ethics washing

Avoid rigid top-down moralism

Support repeatable, contextual decisions

Ethics as a Service as a bounded service block

Why “as a service”?

not because ethics should be outsourced blindly

but because service models clarify responsibility and interfaces

the useful analogy is Platform as a Service

Ethics as a Service as a bounded service block

The compromises behind EaaS

- between too flexible and too strict
- between fully centralised and fully devolved responsibility
- between abstract principles and local implementation

Ethics as a Service as a bounded service block

Core components of EaaS

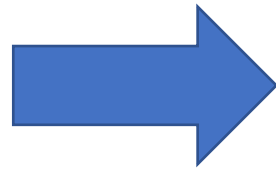
- independent multi-disciplinary ethics advisory board
- collaboratively developed ethical code
- contextual interpretation process
- translational tools
- regular audit and review

Ethics as a Service as a bounded service block

The internal logic of the service block

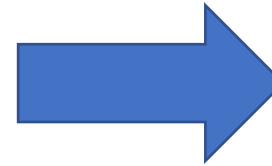
INPUT:

- System proposal
- Context
- Stakeholders
- Risks



PROCESSING:

- Principle interpretation
- Foresight questions
- Tool selection
- Review



OUTPUT:

- Justified recommendations
- Constraints
- Escalation points
- Audit trace

Ethics as a Service as a bounded service block

Digital Catapult framework as an example of layered translation

Level 1:
high-level principles

Level 2:
contextual interpretations

Level 3:
discourse and foresight questions

Level 4: practical tools

Ethics as a Service as a bounded service block

*What this service block **is not***

not a final moral authority

not a static compliance stamp

not a replacement for engineering judgement

not a replacement for law

not a replacement for organisational responsibility

Ethics as a Service as a bounded service block

Workshop 3: design an EaaS service contract

Define an example and then define:

- Service inputs
- Service outputs
- Responsibilities
- Review moments
- Escalation triggers

Ethics as a Service as a bounded service block

Debriefing: What a good EaaS service contract looked like

Example: LLM code-review assistant

Inputs: PR diff, security policy, licence policy, repo scope, risk label

Outputs: flagged issues, required checks, escalate/no-escalate, rationale

Owner: release manager

Review again: after model/prompt/policy change

Escalate: on security, privacy, licence, or unexplained high-risk output

1. Service inputs

decision request
system context
stakeholders / affected groups
applicable policies or principles
evidence artefacts

2. Service outputs

risk judgment
required constraints or mitigations
proceed / revise / escalate
short rationale
trace/provenance record

3. Responsibilities

caller provides complete context
EaaS evaluates and records reasoning
product owner implements follow-up
human owner signs off high-risk cases

4. Review moments

before deployment
after major model/data/prompt changes
at release gates
after incidents or complaints
periodic post-deployment review

5. Escalation triggers

missing context
conflicting principles
failed privacy/fairness/safety checks
low confidence or unstable result
high-risk domain or vulnerable users



Ethics as a Service in Service-oriented Software Engineering

Part 4 – LLM-assisted software engineering and interaction-time governance

Embedded Special Topic for Service-oriented Software Engineering

University of L'Aquila

Patrizio Migliarini, PhD

LLM-assisted software engineering and interaction-time governance

EaaS in the LLM era

LLMs are now part of SE workflows

- many teams do not control:
- training data
- model weights
- provider infrastructure

so governance must focus on what they can control

Migliarini et al. - Ethical Prompt Engineering for AI-driven SE

<https://dl.acm.org/doi/10.1145/3801980>



LLM-assisted software engineering and interaction-time governance

What is interaction-time governance?

- prompts and templates
- policy bindings
- project context bindings
- retrieval scope
- checks and verification passes
- provenance and audit trails

LLM-assisted software engineering and interaction-time governance

The real governance problem is not “write a better prompt”

the real problem is making controls:

- Explicit
- Reviewable
- Replayable
- Auditable over time

LLM-assisted software engineering and interaction-time governance

Why this matters in software engineering

Requirements critique

Design review

Code review

Test generation

Documentation

Maintenance support

LLM-assisted software engineering and interaction-time governance

Core capability groups

- G1 Policy and context handling**
- G2 Prompt planning and patterning**
- G3 Provenance and audit**
- G4 Evaluation and reporting**

LLM-assisted software engineering and interaction-time governance

Workshop 4: govern an LLM invocation

Case: LLM-assisted code review for a high-risk pull request

A pull request modifies:

- authentication logic
- user session handling
- audit logging

The repository belongs to a healthcare platform.

Design the invocation wrapper

Prompt template

role, task, expected output structure, refusal conditions

Policy bindings

security policy, privacy policy, licence policy, risk label, jurisdiction

Retrieval limits

which files, docs, tests, ADRs, and tickets may be retrieved

Checks

security, privacy, citation, licence, and consistency checks

Provenance fields

what must be logged so the invocation is reviewable and replayable

Deliverable

A concrete invocation design that a release team could actually use.

LLM-assisted software engineering and interaction-time governance

Debriefing: what a strong invocation wrapper looked like

Prompt template

- clear role: security-aware code reviewer
- fixed output schema
- evidence required for every claim
- refusal if context is missing

Policy bindings

- bind security policy
- bind privacy constraints
- bind healthcare risk level
- bind applicable jurisdiction
- bind licence/compliance rules

Retrieval limits

- changed files only
- linked tests and ADRs
- relevant tickets only
- no full-repo open retrieval
- no unrelated patient-data artefacts

Checks

- citation coverage
- no secrets or PII in output
- security checklist pass
- policy consistency check
- escalation if evidence is weak or conflicting

Provenance fields

- model and version
- template version
- policy IDs
- retrieved artefacts
- check results
- reviewer decision

LLM-assisted software engineering and interaction-time governance

Transition to ethical interpretation

Governance is not only about constraining outputs

It is also about interpreting morally significant situations

This opens the door to ethical profiling and ethical interpreters



Ethics as a Service in Service-oriented Software Engineering

Part 5 – Ethical interpreters, ethical profiling, and practical examples

Embedded Special Topic for Service-oriented Software Engineering

University of L'Aquila

Patrizio Migliarini, PhD

LLM-assisted software engineering and interaction-time governance

What is an ethical interpreter?

a component that interprets ethically charged scenarios

identifies a moral lens

assesses acceptability

explains the reasoning

Migliarini et al. - Advancing Automated Ethical Profiling in SE

<https://dl.acm.org/doi/10.1109/ASE63991.2025.00202>



LLM-assisted software engineering and interaction-time governance

Why ethical profiling matters

- users have ethical preferences
- those preferences can vary by context
- manual elicitation is costly and incomplete
- automation can help, but must remain auditable

LLM-assisted software engineering and interaction-time governance

Three-theory evaluation as a practical lens

- Utilitarianism
 - Deontology
 - Virtue ethics
-
- one scenario can be viewed through more than one lens

LLM-assisted software engineering and interaction-time governance

What the profiling study found

- non-trivial agreement across models
- stronger agreement on simpler scenarios
- divergence on edge cases
- human oversight remains necessary

LLM-assisted software engineering and interaction-time governance

Example 1: privacy in a travel booking flow

Scenario: user accepts all cookies during purchase

EaaS help:

- identify ethical issue
- classify risk
- require policy trace
- force disclosure and consent review
- escalate if dark patterns are suspected

LLM-assisted software engineering and interaction-time governance

Example 2: code generation with licensing risk

Scenario: LLM generates code for a deadline-critical task

EaaS help:

- bind licence policy
- require source provenance
- run attribution checks
- route risky outputs to human review

LLM-assisted software engineering and interaction-time governance

Example 3: bug triage in a safety-relevant product

Scenario: LLM helps prioritise issues

EaaS help:

- bind severity taxonomy
- expose reasoning factors
- flag fairness and safety concerns
- keep human accountability visible

LLM-assisted software engineering and interaction-time governance

Workshop 5: build a mini ethical interpreter workflow

Case: Should a developer use copied code from an unknown online source in a hospital system?

A developer finds code online that solves a bug quickly.

The source is unclear, the licence is unknown, and the code will be used in a hospital platform.

produce a mini interpreter result

Ethical lens

which lens is most relevant here?

Judgement

acceptable / unacceptable / unclear

Explanation

short reason linked to the chosen lens

What must be logged

scenario, lens, judgement, explanation, evidence, reviewer

When to escalate

what makes human review mandatory?

LLM-assisted software engineering and interaction-time governance

Debriefing: what a strong mini ethical interpreter looked like

Ethical lens

deontology: duty, rules, obligations

consequentialism: likely harms and benefits

virtue ethics: integrity, prudence, responsibility

“Deontology is the main lens because unknown provenance and unknown licence create a duty problem before deployment.”

Judgement

acceptable only with conditions

unacceptable if provenance remains unknown

unclear if evidence is insufficient

“Unacceptable at this stage because the source and licence are unknown in a safety-critical context.”

Explanation

Short, linked to the chosen lens, tied to the software context

not generic moral language

“Using unverified code in a hospital system violates duties of safety and professional responsibility.”

What should be logged

scenario ID

selected lens

judgement

short rationale

evidence used

reviewer / timestamp

“Log the scenario, the deontological interpretation, the rejection reason, the missing licence evidence, and the human reviewer.”

When to escalate

unclear provenance

unknown licence

high-risk domain

disagreement or uncertainty

missing evidence for safe reuse

“Escalate to legal and security review before any integration into production code.”

